

Student No. _____

Name: _____

특별한 언급이 없는 한, 32bit x86 프로세서의 protected mode 에서 MASM 을 사용함을 가정함

1. [10pt] 다음은 Boolean 명령어를 이용하여 주어진 문자열에서 대문자를 소문자로, 소문자를 대문자로 바꾸는 어셈블리어 프로그램의 일부를 보여준다. 빈 칸에 알맞은 명령어 구문을 쓰시오. (Boolean 명령어가 아닌 명령어를 사용할 경우 0 점)

<pre>.data array BYTE "sEOULtECH" .code mov ecx, LENGTHOF array mov esi, OFFSET array L1: [] inc esi loop L1</pre>	Ascii code (10 진수)	Character
	65	A
	66	B
	:	:
	89	Y
	90	Z
	:	:
	97	a
	98	b
	:	:
	121	y
	122	z

[예상 결과]

array에 "SeoulTech"이 저장됨

2. [10pt] CMP 명령어는 두 피연산자에 대한 subtraction으로써 내부적으로 구현되어진다. 따라서, 사용자는 CMP 명령어를 사용한 직후, 특정 flag 값들을 확인함으로써 두 피연산자들의 대소관계를 파악할 수 있다. 두 피연산자가 signed 인 경우와 unsigned 인 경우로 나누어, 두 피연산자들의 관계에 대해 어떤 flag 들이 어떤 값을 갖는지를 확인해야 하는지 설명하시오.

두 피연산자를 signed 로 간주할 경우	두 피연산자를 unsigned 로 간주할 경우

3.(1) [10pt] x86 시스템에서 일반적으로 곱셈은 비싼 연산에 속하지만, Intel 은 최적화를 통해 곱셈의 수행속도를 개선할 수 있었다. Intel 에서 시프트 연산을 통해 곱셈을 최적화했다고 가정한다면, 다음 어셈블리어 프로그램에서 사용된 곱셈이 어떻게 최적화될지 어셈블리어로 구현하시오. (단, IMUL 의 두 피연산자는 UNSIGNED 정수라 가정한다.)

최적화 이전	최적화 이후
<pre>.data var DWORD ? .code mov eax, var imul eax, 72</pre>	<pre>.data var DWORD ? .code mov eax, var mov ebx, eax [] [] []</pre>

3.(2) [5pt] 다음은 x86 시스템에서의 곱셈과 나눗셈에 관한 설명이다. 옳지 않은 설명을 모두 고르시오.

답: _____

- ① MUL 명령은 연산의 결과로써 크기가 2 배 커진 결과값을 생성하므로 Overflow 나 Carry 가 발생할 수 없고, 해당 flag 들도 1 로 세팅되지 않는다.
- ② DIV 명령은 MUL 명령이 수행하는 연산의 역으로 볼 수 있고, dividend(피제수)가 divisor(제수)와 quotient(몫)보다 2 배 크므로 역시 overflow 가 발생할 수 없다.
- ③ 하지만, DIV 명령에서 dividend(피제수)가 0 일 경우, 문제가 발생하므로 DIV 명령을 수행하기 전에 이를 검사하는 것이 좋다.
- ④ MUL 명령은 피연산자를 오직 하나만 갖는데, 상수(immediate value)는 피연산자로 사용될 수 없다.
- ⑤ 피연산자를 2 개 또는 3 개 갖는 IMUL 명령을 unsigned 정수에 대해 사용한다면, MUL 연산을 사용한 결과의 하위 반쪽을 사용하는 것과 동일한 효과를 얻는다.

3.(3) [5pt] x86 시스템은 IDIV 명령을 위한 효과적인 부호 확장 (sign extension) 명령어들을 제공한다. 이 중 하나인 CDQ 를 직접 구현한다 가정할 때, 빈칸에 들어갈 적절한 명령어 구문을 쓰시오.

부호 확장 명령어를 사용한 예	부호 확장을 직접 구현한 예
<pre>.data dwvar1 SDWORD ? dwvar2 SDWORD ? .code mov eax, dwvar1 cdq idiv dwvar2</pre>	<pre>.data dwvar1 SDWORD ? dwvar2 SDWORD ? .code mov eax, dwvar1 mov edx, 0 jz L1 mov edx, L1: idiv dwvar2</pre>

4. [10pt] 다음은 x86 시스템에서의 메모리 관리에 관한 설명이다. 옳지 않은 설명을 모두 고르시오.

답: _____

- ① x86 시스템에서 어셈블리어 응용 프로그램을 작성할 때, 동적 메모리 할당이 필요하다면 운영체제에게 heap 메모리에서의 메모리 블록을 요청할 수 있고, 해당 메모리 블록에 대한 주소를 반환 받는다.
- ② Data 세그먼트상에 존재하는 전역 변수(global variable)와 달리, 동적 할당된 변수와 지역 변수는 stack 세그먼트상에 위치한다.
- ③ 16 비트 세그먼트 레지스터에 저장된 값과 32 비트 offset 을 결합하여 32 비트 linear address 를 얻는데, 이는 paging 을 사용하지 않는다면 메모리에서의 physical address 에 해당한다.
- ④ 운영체제가 paging 을 사용한다면, linear address 는 page translation 과정을 거쳐 32 비트 physical address 로 변환된다. 일반적으로 linear address 보다 physical address 의 address space 가 더 크다.
- ⑤ 하나의 page 내에서 서로 연속하여 위치한 메모리 주소는 실제 physical 메모리 내에서도 연속되어 위치한다.

Student No. _____

Name: _____

5.(1) [10pt] 주어진 두 인자를 뒤바꾸는 swap 함수가 다음과 같이 c 언어로 쓰여져 있다. Swap 함수를 어셈블리어 프로그램에서 호출하고자 하고 C calling convention 이 사용된다 가정하자. C 컴파일러에 의해 Swap 함수가 어셈블리어로 번역된다면 어떻게 작성될지 예상하여 빈 칸을 채우시오. (이 문제에서 enter 와 leave 명령어는 사용할 수 없음)

C 로 작성된 swap 함수	어셈블리어로 번역된 swap 프로시저
<pre>void swap(int * a, int * b){ int temp = *a; *a = *b; *b = temp; }</pre>	<pre>swap PROC </pre> <pre>swap ENDP</pre>

5.(2) [10pt] 5.(1)의 swap 프로시저를 호출하고자 하고 C calling convention 이 사용된다 가정하자. Swap 프로시저가 어떻게 호출되어야 할지 빈 칸을 채우시오.

swap 함수를 호출하는 부분	swap 프로시저를 호출하는 부분
<pre>int x = 1; int y = 2; void main(void){ ... swap(&x, &y); ... }</pre>	<pre>.data x DWORD 1 y DWORD 2 .code swap PROC ... swap ENDP main PROC ... </pre> <pre>... main ENDP END main</pre>

6. 주어진 설명에 맞게 동작하도록 빈칸에 적절한 명령어를 넣으시오.

(1) [5pt]

다음은 MASM의 conditional control flow directive를 이용해 작성된 프로그램이다. 이를 해당 directive를 사용하지 말고 어셈블리 프로그램으로 작성하시오. (단, sum과 eax의 값은 unsigned 정수라 가정)

```
.data
sum DWORD 0

.code
mov eax, 0
.WHILE eax < 10
inc eax
add sum, eax
.ENDW
```

```
.data
sum DWORD 0

.code
mov eax, 0
L1:
[ ]
inc eax
add sum, eax
[ ]
L2:
```

(2) [10pt]

다음 C/C++ 코드를 어셈블리 프로그램으로 작성하시오. (단, VAR1 ~ VAR4는 4byte int형이라 가정함)

$VAR4 = (-VAR1 * 5) / (VAR2 * VAR3)$

```
.data
VAR1 DWORD ?
VAR2 DWORD ?
VAR3 DWORD ?
VAR4 DWORD ?

.code
```

MOV EBX, VAR2

MOV EAX, VAR1

CDQ

MOV VAR4, EAX

7. 다음 어셈블리어 프로그램의 일부를 보여준다. 만약 이 프로그램들이 실행된다면, 주어진 레지스터(또는 FLAG)에 최종적으로 저장될 값을 답하시오(16 진수로 답한다면 suffix 'h'를 붙여 16 진수임을 명확히 하시오). 만약, 어셈블 과정에 에러가 발생한다면, 어느 구문에서 왜 에러가 발생하는지 설명하시오.

(1) [5pt] AL 레지스터

```
.code
MOV AL, 32
SAL AL, 2      ; SHIFT LEFT
SAR AL, 2      ; SHIFT RIGHT
```

답:

(2) [5pt] OF 와 AX 레지스터

```
mov al, 23
mov bl, 6
imul bl
```

답:

(3) [5pt] BL 레지스터

```
.data
var1 BYTE 160
var2 BYTE 128

.code
mov al, var1
add al, var2
mov bl, var1
ja L1
mov bl, var2
```

L1:

답: